UCUG1903 Signals and Information Technology

2026-27 Fall Term, Assignment 3

In this assignment, you will use MATLAB to generate the same melody you chose in Assignment 2 but with a piano-like timbre and generate their corresponding spectrum and spectrogram. Please **upload a single ZIP file** containing your MATLAB scripts, and a Word or PDF report to Canvas by September 30, 2026, at 23:59. Late submissions are accepted but will incur a 50% deduction per day.

Task 1: Harmonics and Hit Envelope (Timbre) [60]

a) Problem description

In this task, you will synthesize **the same melody** you chose in **Assignment 2**, but instead of using pure tones, you will generate each note with a **piano-like timbre**. To achieve this, you will:

1. Add harmonics – Superimpose multiple sinusoidal overtones (integer multiples of the fundamental frequency) to enrich the sound. Include harmonics up to the 5th order, with amplitudes that decay as the harmonic order increases (e.g., [1.00, 0.45, 0.25, 0.12, 0.06] for the 1st through 5th harmonics).

2. **Apply an amplitude envelope** – Shape the volume of each note over time (e.g., a quick attack followed by a gradual decay) to mimic the natural behavior of a struck piano string.

b) Implementation instructions

Step 1: Generate one harmonic tone

Open the file generate_harmonic_tone.m and complete the function to generate a harmonic tone containing up to the 5th harmonic. This process is similar to generate_tone; however, in addition to the fundamental frequency signal, you must create signals at 2×, 3×, 4×, and 5× the base frequency. Weight and sum these components together using the coefficients defined in the harmonic_amps array.

Hints:

- You can use a for loop to iterate over the harmonics and avoid repetitive code.
- For signal summation, you could reuse the signal_add function created in Assignment 1.

Step 2: Implement the hit envelope

Open the file generate_piano_envelope.m and, given num_samples, return a vector envelope of that length representing the amplitude envelope. To do so, define attack_len and decay_len such that they sum to num_samples, then build the attack phase scaling linearly from 0 to 1 and the decay phase scaling from 1 to 0, and concatenate them. A piano like envelope typically features a quick attack, approximately 5% of the total duration, followed by an long-tail decay.

Hints:

- For the attack, a linear change suffices, while for the decay you can use a squared or exponential curve; a squared decay can be generated by creating a linear scale and then squaring each point.

linspace

Syntax

y = linspace(x1,x2)

y = linspace(x1,x2,n)

Description

y = linspace(x1,x2) returns a row vector of evenly spaced points between x1 and x2. By default, linspace generates 100 points.

y = linspace(x1,x2,n) generates n points. The spacing between the points is (x2-x1)/(n-1).

linspace is similar to the colon operator, ":", but gives direct control over the number of points and always includes the endpoints. "lin" in the name "linspace" refers to generating linearly spaced values as opposed to the sibling function logspace, which generates logarithmically spaced values.

Step 3: generate the timbre melody

Open play_melody_timbre.m and first generate your chosen melody using only the harmonic tones (without the envelope), then generate the same melody using both the harmonic tones and the piano envelope

Hints:

- When you apply the envelope on tones with harmonics, you can use signal_multiply build in assignment1.
- For the two steps, you should be able reuse most of the code.

The provided output-generation code will generate two audio file at output and plot a portion of the waveform. Also a plot of eveolp you created. In your submission, include the generated WAV file. **In your report, include a screenshot of the plot, state which song you chose, and briefly describe what you hear, how does the waveform change and envelope looks like.**

Task 2: Generate Spectrum and Spectrogram [40]

a) Problem description

In this task, you will convert the melody you generated in Task 1 into the frequency domain. To do this, you may use two built-in MATLAB functions: fft() and spectrogram().

The fft() function computes the fast Fourier transform (FFT), which is the most common method for converting a time-domain signal into the frequency domain. The details of how it works, or why it works, are not covered in this course. For this assignment, you only need to quickly examine what it produces and how the output changes with the input and function parameters. The spectrogram() function can be viewed simply as applying fft() with

a sliding window across the time domain, or mathematically, as the short-time Fourier transform (STFT). You also only need to understand how to use it.

Following is the syntax for fft. **In this course, you only need to understand this realization. Other realizations are supported, but they are not covered.**

Currently, you can think of X as the piece of signal you want to convert to the frequency domain. Remember the time-interval issue discussed in class: the length of the section affects the final spectrum. Therefore, please experiment with the length of X to see how the spectrum changes. You should check the spectrum of the full melody and of different notes.

fft Fast Fourier transform

- Syntax

  Y = fft(X)

- Description

  Y = fft(X) computes the discrete Fourier transform (DFT) of X using a fast Fourier transform (FFT) algorithm. Y is the same size as X.If X is a vector, then fft(X) returns the Fourier transfo rm of the vector.

Similarly, for spectrogram, you only need to understand the following two realizations.

The first format gives you a 2D histogram with some default settings. In the second format, you can specify the window used for spectrum generation, the number of samples between adjoining windows, and freqSpec. freqSpec has two meanings: if it is a number, then it is the number of FFT points for each window, and it should be a power of 2, such as 128 or 256. If it is a vector, then each value of the vector is a frequency number, and the function will return the amplitude of the corresponding frequency in a vector. Here, you can simply remember this without fully understanding it. You can try changing the parameters and see what difference they make. Some observation and explanation, or observation only, is acceptable.

Spectrogram Spectrogram using short-time Fourier transform

- Syntax

  s = spectrogram(x)

  [s,f,t] = spectrogram(x,win,nOverlap,freqSpec)

- Description

  s = spectrogram(x) returns the STFT of the signal x. Each column of s contains an estimate of the short-term, time-localized frequency content of x.

  [s,f,t] = spectrogram(x,win,nOverlap,freqSpec) returns the STFT of the signal x and the frequencies f (rad/sample) and sample indices t. The spectrogram function:

  Uses win to divide the signal into segments and windows them.

  Uses the overlap length specified in nOverlap to overlap samples between adjoining segments.

  Computes the discrete Fourier transform of each windowed segment over the number ofpoints or at the frequencies specified in freqSpec.

b) Implementation instructions

Open the frequency_analysis.m file. In this section, most of the function code is already provided. **You only need to write code to:**

- Compare the different inputs with and without the envelope. For this, you just need to change the file that is read in.
- Read different sections of your melody to obtain the corresponding spectrum. Here, you may need to write several lines of code to specify the start and end points of the melody fraction, or directly specify the index of the section. Think about how to calculate this based on your sampling rate and note duration.
- The spectrogram code already provides full functionality. You can experiment with the window length in samples, the number of overlapping samples between adjoining windows, and the FFT resolution.

Please consider questions such as: What does the spectrum look like for these two inputs? Are they quite different? What about the spectrogram? Why?